A job ready bootcamp in C++, DSA and IOT

Recursion



Saurabh Shukla (MySirG)

Agenda

1. What is a recursion?
2. Recursion Tree | Tracing code
3. How to approach recursive solution?
4. Few examples

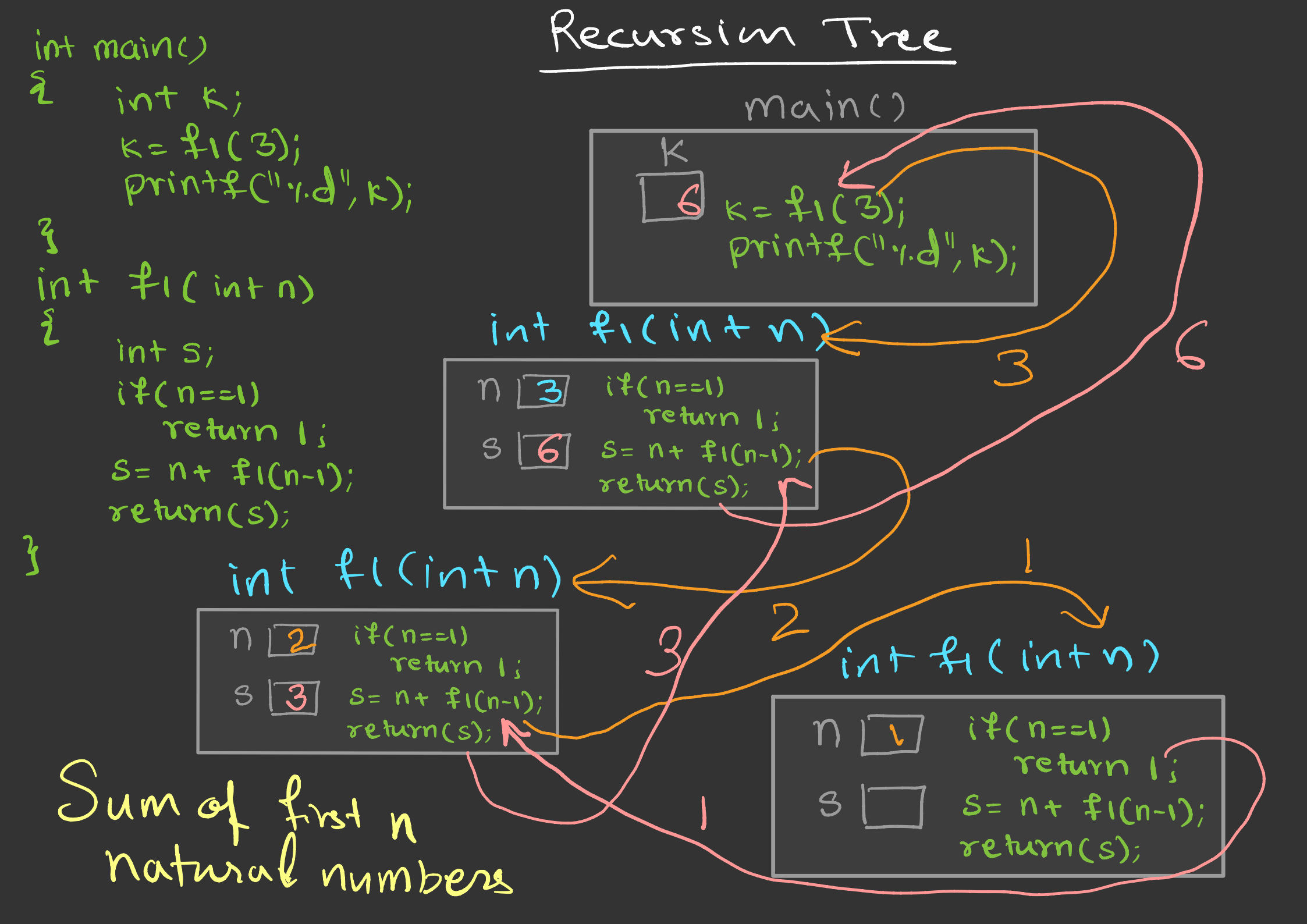
What is a Recursion?

- Function calling itself is called recursion.
- A recursive method solves a problem by calling a copy of itself to work on a smaller problem.
- It is important to ensure that the recursion terminates.

- Each time the function call itself with a slightly simpler version of the original problem.
- Recursive code is generally shorter and easier to write than iterative code.
- Solution to some problems are easier to formulate recursively.

Recursion Tree

```
int main()
{
    int k;
    k = f1(3);
    printf("%d", k);
}
int f1(int n)
{
    int s;
    if(n==1)
        return 1;
    s = n + f1(n-1);
    return(s);
}
```



Sum of first n natural numbers

How to approach a Recursive Problem?

Write a recursive function to calculate sum of first n natural numbers.

```
int sum(int n)
{
    if(n==1)
        return(1);
    return n+sum(n-1);
}
```

① sum(n) 1+2+3+4+...+n

RC ② n+sum(n-1) 1+2+3...+n-1

BC ③ n>=1 recursion

n==0 recursion stops)

n==1 return 1

Sum(n) = 1+2+3+4+...n-1 +n

Sum(n-1) = 1+2+3+4+...n-1

15 sum(5);

10
5 + sum(4)

6
4 + sum(3)

3
3 + sum(2)

1
2 + sum(1)

1

return 1 ⟶ n == 1

return n + sum(n-1) ⟶ otherwise

Write a recursive function to calculate factorial of n.

```
int fact( int n)
{
    if(n==0 || n==1)
        return 1;
    return n*fact(n-1);
}
```

① ② RC ③ BC

fact(n) $1 \times 2 \times 3 \times \ldots \times n$

$n \times$ fact(n-1) $1 \times 2 \times 3 \times \ldots \times (n-1)$

0! or 1! 1

n! $n*(n-1)!$

Write a recursive function to print first n natural numbers.

```
void printN(int n)
{
    if(n>0)
    {
        printN(n-1);
        printf("%d", n);
    }
}
```

① printN(n) 1 2 3 4 ..... n

RC ② printN(n-1) 1 2 3 4 ... n-1

② printf("%d", n);

BC ③

{ n==0 BC

n>0 RC